

DocProcessor

DocProcessor

Tagline: Turn documentation into a verifiable feature map for QA automation.

Summary: DocProcessor is a standalone, fully decoupled Go module that loads project documentation, builds structured feature maps, and tracks verification coverage. It is designed to work with LLM agents for intelligent feature extraction but also includes heuristic extraction for offline use.

Short description (~40 words): A project-agnostic Go module for documentation processing and feature-map extraction. It parses docs into structured feature maps and tracks which features are verified — using LLM agents for intelligent extraction or heuristics offline — feeding QA automation with an anti-bluff, always-matches-reality guarantee.

Long description (150-250 words): DocProcessor bridges documentation and QA. Given a project's documentation, it builds a structured feature map — an enumerated model of what the product claims to do — and tracks verification coverage against it, so teams can see which documented features are actually proven by tests and which are not. It is deliberately dual-mode: it works with LLM agents for intelligent, semantic feature extraction when available, and falls back to heuristic-based extraction for fully offline use, so it never hard-depends on a model being present. Architecturally it is a standalone, project-not-aware, fully decoupled Go module (CONST-051(B)): it ships zero project-specific values and is incorporated by consumers as an equal-codebase submodule. Its documentation and claims are held to the project's anti-bluff covenant (CONST-035) and full-automation coverage rules (CONST-048): every capability the README advertises is exercised by an automated test or Challenge script in the repository, matching an explicit operator mandate that tests and Challenges must confirm real, end-user-usable behavior rather than merely executing successfully. User-facing strings are localized through the CONST-046 i18n translator seam. In practice, DocProcessor is the input side of the QA loop that HelixQA closes: DocProcessor extracts the feature map from docs,

and the QA orchestration proves each mapped feature with runtime evidence — making documentation, tests, and shipped behavior converge instead of drift.

Why we built it: Documentation and tests drift apart: docs promise features that no test proves, and QA can't easily tell what "complete" means. DocProcessor turns docs into a machine-readable feature map so verification coverage can be measured against what was actually promised.

Why it's a game-changer: It makes "is this documented feature actually verified?" a queryable, automatable question, and it does so without hard AI dependencies — LLM extraction when available, heuristics when not — so it runs anywhere.

What's innovative: - Documentation-to-feature-map extraction with verification-coverage tracking. - Dual extraction: LLM-agent-driven or heuristic/offline. - Project-agnostic, zero-config decoupling (CONST-051(B)). - Anti-bluff self-verification: README claims are backed by tests/Challenges (CONST-035/048).

Biggest technical challenges + how solved: - *Model-optional operation*: solved with a heuristic extractor fallback so the module works offline. - *Keeping docs and reality aligned*: solved by structured feature maps + verification coverage tracking, tied into the QA loop. - *Reusability*: solved via strict decoupling and equal-codebase submodule consumption. - *Credibility of its own claims*: solved with anti-bluff tests/Challenges for every advertised capability.

Tech stack (why + how): - **Go (1.25+)** — module core; Apache-2.0 licensed. - **LLM agents** — intelligent semantic feature extraction (optional). - **Heuristic parser** — offline feature extraction fallback. - **i18n Translator (pkg/i18n)** — CONST-046 localized strings. - **Challenge harness** — anti-bluff verification of the module's own claims.

Public links: - GitHub (HelixDevelopment): <https://github.com/HelixDevelopment/DocProcessor> (public). - GitHub (vasic-digital): <https://github.com/vasic-digital/DocProcessor> (public). - Consumed with HelixQA in the QA-automation pipeline.

Suggested diagrams/illustrations (OpenDesign): 1. Docs → feature map → verification-coverage matrix. 2. Dual-extractor switch: LLM path vs heuristic/offline path. 3. QA loop: DocProcessor (extract) → HelixQA (prove with evidence) → convergence. 4. Coverage dashboard concept (documented vs verified features).

Site relevance: vasic.digital (QA-automation / AI-tooling depth). Pairs with HelixQA in the "anti-bluff" narrative.

Priority tier: vasic-util-secondary

Source provenance: `gh repo view vasic-digital/DocProcessor`
README (overview, dual extraction, decoupling, anti-bluff
mandate, i18n, license, Go version); `_analysis/github-helix-`
`others.md` and `_analysis/github-vasic-digital.md` (listings, size/
description).